

CSE 144 Final Project Report

Transfer Learning Challenge

Sam Vo 2141433

AJ McDaniel 2246154

Jarod Cardenas 1903566

Spring 2026

I. Introduction

The objective of this project is to apply transfer learning to train an image classifier to classify images into 100 categories. Given that the provided training dataset only contained 10 images per class, training a full deep neural network would lead to severe overfitting. To resolve this, transfer learning utilizes a pretrained model by keeping feature extractors relevant to our classifiers. Only the top layers are retrained, allowing us to adapt models originally trained on much larger data sets for our smaller application.

We initially started using Convolutional Neural Networks, specifically smaller RegNet_Y and EfficientNet models, to establish a baseline. CNNs focus only on nearby pixels, which limits their ability to see the bigger picture of an image, along with how max pooling layers and downsampling discard spatial details that affect classification. Vision Transformers, however, use self attention to analyze the entire image at once, helping them understand complex patterns more effectively at the cost of compute resources.

Ultimately we decided to use a multi model ensemble by aggregating the predictions of multiple vision transformer models, we were able to average out individual model biases and prediction noise. Initially, AJ reached 87% accuracy using a 5-model, sparsely-trained DINOv3 ensemble, but we realized that aggressive data augmentations like CutMix may be introducing too much noise and that the ensemble required more diversity. Sam combined three different model architectures, DINOv3, ViT_L_16, and RegNet_Y_16GF, with Test Time augmentation. The design that achieved our final 94.54% accuracy was a 2 model ensemble of our strongest single model; SigLIP2 + SigLIP's text encoder, and Facebook's Perception Encoder.

II. Dataset

The data set consists of 100 total classes with ~10 training images per class – 1079 in total – and 1036 unlabeled test images for submission validation. We noticed that the training data groups these classes into specific categories ('domains'): 0-29 represent food, 30-49 denote flowers, 50-69 encompass cars, and 70-99 cover aircraft.

The dataloader executes and initially resizes incoming images to a square 384 pixels, allowing random cropping functions to extract distinct 336 pixel patches without losing relevant pixels in the image. This is followed by pixel intensity normalization across channels using standardized ImageNet statistics to stabilize the calculation of error gradients. Given the constraint of ~10 images per class (even as little as 4 in the case of label 42), the pipeline subjects the training data to aggressive data augmentation including: horizontal flips, rotations, grayscale conversion, and color jitter. CutMix was also used during later phases of training which helped earlier

submissions improve accuracy. While trying to apply CutMix earlier on in training, it caused too much noise and was susceptible to overfitting.

III. Implementation

To adapt Facebook's Perception Encoder to output 100 integer classes, the default classification head was removed in order to access the terminal global average pooling layer. To preserve the feature space the entire internal Vision Transformer backbone was frozen. Turning off gradient tracking during the forward pass saves memory by preventing unnecessary background calculations. A similar method was used to implement SigLIP2 and its text probabilities; added for ensemble diversity.

Appended to the exposed backbone was an initial layer that featured a dropout instance with a masking probability of 0.4. This stochastic regularization randomly zeroes out incoming features during training, preventing the network from heavily weighting specific parameters and forcing it to map a distributed consensus. A standard fully connected layer then projected the high dimensional feature vector down to the 100 required classes.

The custom classification head was optimized using categorical Cross Entropy Loss, which handles the smoothed probability distributions generated by the data augmentation steps. Gradient descent was managed by the Adam optimizer, restricted exclusively to the unfrozen parameters. A Cosine Annealing learning rate was systematically reduced from 0.001 to near zero across epochs.

Initial architecture was executed locally on Apple Silicon, utilizing 32GB LPDDR5X of RAM to load massive models and later to a desktop system with an RTX 3090 24GB of GDDR6X VRAM. Although the desktop system has less ram for loading models, it supports up to 936 GB/s bandwidth and more processing power. To accelerate the final ensembling phase, heavy training workloads were migrated to the desktop environment allowing for simultaneous model caching and increased batch sizes.

IV. Experiments

Initial testing established a baseline using the RegNet_Y_128GF CNN, but quickly discovered that the high parameter count would overfit our tiny training dataset. This caused us to look to other models such as EfficientNet-VS-2 which gave us our first submission result of ~69% (after addressing overfitting).

To validate and improve final performance, the inference protocol relies on Test Time Augmentation and deep ensembling. During inference, the pipeline generates augmented variations of each test image. Test Time Augmentation grants the models multiple spatial perspectives of each image helping improve accuracy for visual noise or orientation shifts. By averaging the results together, the models cover for each other's bias and produce a single, (theoretically) more accurate prediction.

Hyperparameter ablation tested various augmentations, freezing strategies, and learning rates, revealing the extreme sensitivity of the frozen backbone pipeline. Applying CutMix simultaneously while unfreezing internal attention blocks introduced too much noise, preventing the loss function from converging properly. The optimal pipeline restricted CutMix exclusively to later epochs once our classification head had stabilized. Additionally, it was determined that an aggressive rate was ideal for training the new head, while a highly conservative rate was strictly required when unfreezing backbone layers (when we decided to do so) to avoid destroying pre learned features.

So as to not gather results blindly, we also stratified-split our results 80/20 per class. This gave us a validation set of 216 images and although per-label results were noisy, the per-domain results drove development and gave us insights into what the model was succeeding (and failing) at.

Ensemble size and membership was also tweaked; the DINOv3 5 model ensemble lacked diversity as although sparsely-trained for variance, they still remained the same model just on different seeds. The next submission of the three-model ensemble of DINOv3, RegNet_Y_16GF, and ViT_L_16 offered better stability and accuracy, solidifying that diversity in the ensemble is what we needed. The idea is that it averages out misses – what one model misses, another may pick up on. The final ensemble of SigLIP2 + text encodings and PE proved to be our most fruitful, but after submitting twice with two different sets of weights to each member's predictions in the ensemble, we received the same accuracy of 0.945454.

V. Results

The model performance continued to improve as we tweaked our baseline. Initially starting with RegNet_Y_128GF we experienced overfitting, leading us to implement with EfficientNet raising our accuracy to 63% and 69% later on. After hitting a wall with CNNs we explored Vision Transformers using a DINOv3 in a 5 model ensemble and increased our accuracy to 87%. Iterating on this design the ensemble count was lowered to 3 and the ensemble consisted of DINOv3, ViT, and RegNet_Y_16GF, with the goal of combining the global reasoning of Visual Transformers and the inductive biases of CNNs. However, this would not prove as accurate as having our final ensemble, scoring 94.5454% submission accuracy.

An analysis of the validation results revealed differences in accuracy across different categories. The model excelled when identifying food and flowers, demonstrating the pretrained models strengths. On the other hand, cars and airplanes were a persisting challenge to our model. Ensembles prior to our final version stalled at a validation accuracy of between approximately 85-87% for automobiles, and 75-77% for aircraft. In the final ensemble, SigLIP alone scored 97.6% validation accuracy in cars and 86.4% in aircraft during per-domain accuracy coalescing, with PE scoring similarly, having minor per-class disagreements in the aircraft category.

This suggests that our lack of training data was unable to fully map all the categories, even with various data augmentation artificially increasing our training set. Increasing model size and quality, as well as ensembling/"Mixture of Experts" with diversity significantly and directly raised our label confidence across all domains.

VI. Discussion

The strategy that worked best was our two-model ensemble of SigLIP2 + text encoding, and the Meta Perception Encoder – each with a frozen backbone paired and a strict classification head. The Perception Encoder by Facebook was trained to align visual patches with text, it provides a robust feature space out of the box. Alternatively, attempting to unfreeze the internal blocks while applying data augmentations like CutMix introduced too much noise.

A major limitation of the current pipeline is the performance on airplanes – whilst significantly better than earlier experimentations, there is still a decent amount of upside to optimize for. A concrete next improvement would be to integrate pseudo labeling to expand the training data. The system could run inference on the unlabeled test images, assign them pseudo labels to highly confident predictions, and then retrain the classification head using this artificially increased dataset. However, this may only increase confidence in all other domains except aircrafts as aircraft accuracy is not above 90% – this method may bake some errors confidently into our model. Therefore, pseudo-labeling with multiple-model gating (add to the training set if multiple models agree on the label) may be worth investigating. One final note is that there may be one more pre-processing optimization to make: cropping to square on aircrafts when they're often the full length of a rectangular image may possibly remove features critical to properly identifying them. SigLIP NaFlex or squashing the image rather than cropping may address this issue and is also worth investigating.

VII. Reproducibility

- 1. Random seeds and determinism settings:
 - SEED = 52

- random.seed(SEED)
- np.random.seed(SEED)
- torch.manual_seed(SEED)
- Torch.backends.cudnn.deterministic = True
- 2. Package versions / environment setup steps:
 - Python 3.12, torch, torchvision, timm, pandas, numpy
 - Pillow, matplotlib, transformers, ~sentencepiece, huggingface_hub, kaggle
- 3. Exact commands to train and to generate submission.csv:
There are no command line commands.
 1. Follow [README.md](#) instructions to setup the virtual environment and download necessary dependencies.
 2. Generate a token from huggingface.co to get access to SigLIP2 model.
 3. Open pe_single_h52.ipynb and run all cells
 4. Open project-SigLIP2-head.ipynb and run all cells

VIII. Team Contributions

- AJ McDaniel: Managed the advanced model implementation (including SigLIP, DINOv3, and Facebook's Perception Encoder), engineered the final multi-model and multi-seed ensemble architectures, executed the cloud compute training runs, and managed the team's version control and code on GitHub. Minor contributions to the report.
- Sam Vo: Designed the initial baseline training pipeline (RegNet_Y_8GF/16GF/128GF, EfficientNet-VS-2), beta testing 'fine-tuning' feasibility via unfreezing backbone layers, coalescing class and domain specific weaknesses ('domain' referring to each of the four image types that we noticed existed in the training data; cars, flowers, food, and airplanes), evaluated the early RegNet and Vision Transformer pipelines, and implemented the Test Time Augmentation (TTA) inference protocol. Minor contributions to the report.
- Jarod Cardenas: Environment debugging with Torch (with the smaller models, it was easier to run locally however we ran into issues getting our scripts to run on 'mps' for MacOS) and preprocessing insights. Authored a majority of the report.

IX. References

- 1. TorchVision documentation / pretrained model sources:
 - Perception Encoder:
 - Model ID: vit_pe_core_large_patch14_336.fb
 - Platform: Loaded into notebook via 'timm'

- Paper: <https://arxiv.org/abs/2504.13181>
- SigLIP 2:
 - Model ID: vit_so400m_patch14_siglip_378.webli
 - Platform: huggingface.co
 - Paper: <https://arxiv.org/abs/2502.14786>
- 2. <https://github.com/AJMCS/CSE144-Final-Project>